ESP32-BASED EMBEDDED FIRMWARE FOR CLOSED-LOOP TEMPERATURE CONTROL OF A COMPACT THERMAL CHAMBER

Gabriel Pereira[1]

[1]Universidade Paulista – UNIP, Computer Engineering, São José dos Campos - SP, Brazil
RA: G8659C1
Project repository: `https://github.com/GMarquesx/PIDlampada`

ABSTRACT

Microcontrollers are the core of most embedded control systems, concentrating acquisition, processing, actuation and communication in a single low-cost device. This paper describes the firmware of an ESP32 microcontroller that controls the temperature of a compact, battery-powered thermal chamber heated by automotive halogen lamps. The firmware uses the ESP32 12-bit ADC to read an LM35 sensor and the battery voltage, the LEDC peripheral to generate a 500 Hz, 10-bit PWM signal that drives the lamps through a MOSFET, and a proportional-integral (PI) algorithm with anti-windup and battery-voltage compensation. Two operating modes are implemented: a bench mode with Wi-Fi and a web interface, and an embedded mode in which the microcontroller stays in light sleep between control updates to extend battery life. Results obtained from datasheet-based estimates and a closed-loop simulation show a battery-side current of 62 mA in bench mode and 3.8 mA in embedded mode, which extends the runtime of the electronics alone from 1.6 to 26 days (16×) on a 3000 mAh pack. With the lamps holding 40 °C, the system runtime increases by 13%, because the heater dominates the energy budget. In both modes the temperature settles within ±0.5 °C of the setpoint in less than 15 min.

Key words *– Microcontrollers, ESP32, Embedded firmware, PWM, Low-power design.*

1. INTRODUCTION

A microcontroller integrates a processor, memory and peripherals such as analog-to-digital converters, timers and communication interfaces in a single chip, which makes it the natural choice for small embedded control systems. Modern devices such as the ESP32 add wireless connectivity and several low-power modes [1, 2], allowing the same hardware to be used both as a laboratory instrument and as a battery-powered product.

Closed-loop temperature control is a good case study for these resources: the microcontroller must sample an analog sensor, run a control algorithm, generate a power signal for the actuator and manage its own energy consumption [3]. This paper presents the firmware of an ESP32 that controls a compact thermal chamber heated by three automotive W5W lamps. The emphasis is on how each microcontroller peripheral was used and on the design decisions about timing and power consumption.

2. MATERIAL AND METHODS

2.1. System overview

The chamber is an acrylic box of approximately $100 \times 100 \times 80$ mm with three 12 V/5 W W5W lamps on the lid and an aluminum plate as thermal mass. An ESP32 DevKit v1 board is the only processing element: it reads the temperature and the battery voltage, computes the control action and drives the lamps through an IRLZ44N logic-level MOSFET [4], as shown in Figure 1. The system is powered by a 3S 18650 battery pack, with a buck converter supplying 5 V to the board.

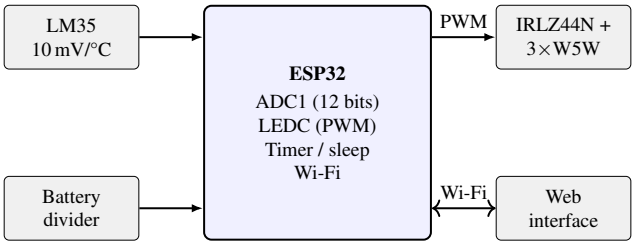


Figure 1: Microcontroller-centered architecture of the thermal chamber.

2.2. Analog acquisition

The LM35 sensor outputs 10 mV/°C [5] and is connected to an ADC1 channel configured with 0 dB attenuation (suggested input range 100–950 mV, i.e. 10–95 °C), which gives a resolution of about 0.27 mV, or 0.03 °C, per code. ADC1 was chosen because, on the ESP32, ADC2 cannot be used while the Wi-Fi driver is active [2]. Each reading is the average of 16 consecutive conversions, which reduces noise without a hardware filter. The battery voltage, needed for the power compensation, is read through a 47 kΩ/10 kΩ resistive divider on a second ADC1 channel with 11 dB attenuation. For a 3S pack (9.0–12.6 V) the voltage at the pin stays between 1.58 and 2.21 V, inside the suggested 150–2450 mV range of that attenuation [6], and a 100 nF capacitor at the pin absorbs the sampling transients of the 8.2 kΩ source impedance. The resolution is about 5 mV per code at the battery, far finer than Eq. (2) requires; the divider drains about 0.2 mA continuously.

2.3. PWM generation

The lamps are driven by the LEDC (LED control) peripheral configured at 500 Hz with 10-bit resolution (1024 duty levels). The low frequency keeps the MOSFET switching losses small, and the thermal inertia of the chamber

makes the PWM ripple irrelevant to the temperature. In embedded mode the LEDC timer is clocked by `RTC8M_CLK` (`LEDC_USE_RTC8M_CLK`) in low-speed mode, the source that the ESP-IDF documentation lists as compatible with light sleep [6], so that the lamps keep receiving the PWM signal while the processor sleeps. At 500 Hz and 1024 levels the timer needs a 512 kHz tick, obtained from the ≈8 MHz oscillator with a divider of about 16.

2.4. Control algorithm

At each control period the firmware computes the error between the setpoint and the measured temperature and updates a PI controller [7]:

$$u = K_p\, e + K_i \int e\, dt \tag{1}$$

The integral term is frozen while the output is saturated (anti-windup). Because the lamp power is proportional to the square of the supply voltage, the duty cycle is corrected by the measured battery voltage:

$$D_{corr} = D_{PI} \left(\frac{12}{V_{bat}} \right)^2 \tag{2}$$

2.5. Operating modes and power management

The firmware has two modes. In *bench mode*, Wi-Fi is on, the loop runs every 1 s and a web page shows the setpoint, temperature and duty cycle in real time. In *embedded mode*, Wi-Fi is off and the ESP32 enters light sleep between control updates, taken every 60 s. Light sleep was chosen instead of deep sleep because deep sleep turns off the digital peripherals and would stop the PWM output, whereas light sleep preserves the peripheral state and resumes execution where it stopped [2].

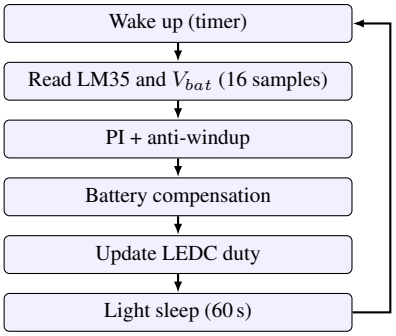


Figure 2: Main loop of the firmware in embedded mode.

2.6. Evaluation procedure

The results of Section 3 were obtained by estimation and simulation, not by bench measurement.

Current. The chip current in each mode was taken from the ESP32 datasheet [1]: 110 mA in bench mode (Wi-Fi receiving and listening, 95–100 mA, plus transmission bursts), 45 mA in embedded mode awake (CPU at 240 MHz, radio off; datasheet range 30–68 mA) and 0.8 mA in light sleep. The DevKit board adds about 6 mA: roughly 5 mA of typical quiescent current of its AMS1117-3.3 linear regulator [8] and about 1 mA of the power LED. The battery-side current was computed from the 5 V board current with an 85% buck-converter efficiency at the nominal 11.1 V of the pack, plus the 0.2 mA of the voltage divider. In embedded mode the processor is awake for 10 ms per cycle (wake-up, 32 conversions, control computation and LEDC update).

Battery runtime. The usable capacity was taken as 80% of a 3S1P pack of 3000 mAh cells (2400 mAh), and the runtime as this capacity divided by the average battery current, with the lamps either off or holding 40 °C.

Closed-loop response. The loop of Figure 2 was simulated in discrete time (Python, fixed random seed) with $K_p = 0.25\,\mathrm{K}^{-1}$ and $T_i = 400$ s ($K_i = K_p/T_i$), including anti-windup, battery compensation, a 10-bit duty cycle and ADC quantization and noise (12 bits, 1.1 V full scale, 4 mV rms per conversion before the 16-sample average). The chamber was represented by two thermal nodes: the aluminum plate (110 J/K), heated by the lamps, and the air and walls (150 J/K), coupled by 1.2 W/K and losing heat to a 25 °C ambient through 0.32 W/K. These values were estimated from the geometry and materials and give an overall time constant of about 13.5 min. The LM35 was modeled with a 10 s time constant and the pack with 11.7 V open-circuit voltage and 0.15 Ω internal resistance. Both modes were simulated for a step from 25 to 40 °C over 60 min, with a control period of 1 s (bench) and 60 s (embedded).

3. RESULTS

All values in this section are estimates or simulation results obtained as described in Section 2.6.

Table 1: Estimated current consumption in each mode, lamps off (mA). Battery: current drawn from the 11.1 V pack.

Mode	Chip	Board (5 V)	Battery
Bench mode (Wi-Fi on)	110	116	61.7
Embedded, awake	45	51	27.2
Embedded, light sleep	0.8	6.8	3.8
Embedded, average	–	–	3.8

Table 2: Simulated closed-loop performance for a 25 to 40 °C step, and estimated runtime (3000 mAh pack, 80% usable).

	Bench (1 s)	Embedded (60 s)
Time to 90% (min)	6.3	6.2
Excursion after rise (°C)	+0.02 / −0.62	+0.20 / −1.15
Settling ±0.5 °C (min)	12.6	14.7
Std. dev. at 40 °C (°C)	0.012	0.087
Mean duty at 40 °C (%)	34.7	35.0
Runtime, lamps off (days)	1.6	26.3
Runtime, 40 °C hold (h)	5.0	5.7

Table 1 shows the estimated currents. In bench mode the board draws about 116 mA from the 5 V rail (62 mA

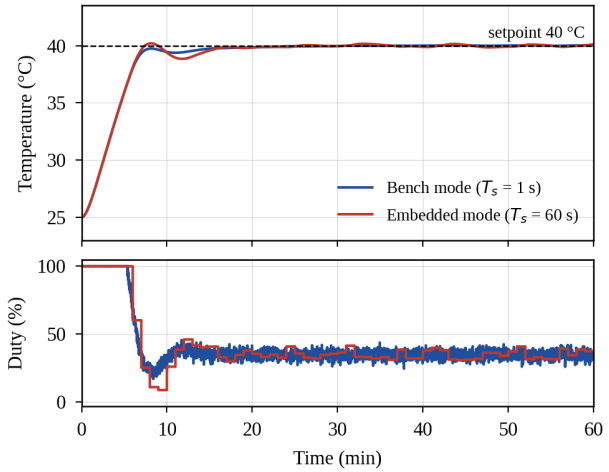


Figure 3: Simulated chamber temperature and PWM duty cycle in closed loop, for a step from 25 to 40 °C.

from the battery), mostly because of the Wi-Fi radio. In embedded mode the average battery current falls to 3.8 mA, a 16-fold reduction, because the processor is awake for only 0.017% of each 60 s cycle and the light-sleep current dominates the average. Of the 6.8 mA drawn by the board in light sleep, only 0.8 mA is consumed by the ESP32 itself; the remaining 6 mA belongs to the regulator and LED of the development board. With 2400 mAh usable, the runtime of the electronics alone (lamps off) increases from 1.6 to 26.3 days (Table 2).

Figure 3 shows the simulated step response. The lamps stay at full duty until about 6 min, when the PI controller takes over, and in both modes 90% of the step is reached after about 6 min. After the rise, bench mode has practically no overshoot (+0.02 °C) and a small undershoot (−0.62 °C), and settles within ±0.5 °C after 12.6 min. With the 60 s update the excursions grow to +0.20 and −1.15 °C and the settling time to 14.7 min. Between 30 and 60 min the standard deviation of the temperature is 0.012 °C in bench mode and 0.087 °C in embedded mode (maximum deviations of 0.03 and 0.16 °C). The mean duty at 40 °C is about 35%, equivalent to roughly 420 mA drawn by the lamps, which is why the runtime holding 40 °C increases only from 5.0 to 5.7 h (+13%) when the electronics change from bench to embedded mode.

4. DISCUSSION

In processing terms the ESP32 is oversized for this task: the loop needs two ADC channels, one PWM output and a few floating-point operations every 1 to 60 s, and the processor is idle for more than 99.9% of the time in embedded mode. A smaller microcontroller would suffice, but the integrated Wi-Fi is what makes the bench mode and its web interface possible without extra hardware, and light sleep with retained peripheral state is what makes the embedded mode practical. The estimated budget also shows that the chip is not the limiting element of the consumption: in light sleep the board overhead (6 mA) is more than seven times the chip current (0.8 mA).

Regarding the ADC, the LM35 signal (250–400 mV for 25–40 °C) lies near the lower limit of the suggested range of the 0 dB attenuation (100–950 mV), where the ESP32 ADC is known to deviate most from an ideal transfer curve, and Espressif notes that the converter is sensitive to noise [6]. Averaging 16 conversions attacks the random noise, but not the systematic error. Since the integral action regulates the measured value, a static ADC error appears directly in the real chamber temperature: at 10 mV/°C, an error of 20 mV, for example, corresponds to 2 °C, far above the 0.1 °C regulation error of the simulation. The figures of Table 2 therefore describe regulation and repeatability, not absolute accuracy, which would require the ADC calibration of ESP-IDF or a two-point calibration against a reference thermometer. The effect on the battery compensation is much smaller, because Eq. (2) only needs the voltage to within a few percent.

The 60 s control period was a reasonable trade-off for this chamber. Its overall time constant of about 13.5 min is 13 times the period, and the price is moderate: the excursion after the rise goes from 0.6 to 1.2 °C, the settling time grows by 2.1 min and the steady-state standard deviation from 0.012 to 0.087 °C, in exchange for a 16-fold reduction of the current of the electronics. At system level, however, the gain is only 13%, since the lamps draw more than 99% of the battery current in embedded mode. A shorter period during the heating transient and 60 s at steady state would improve the response at a small energy cost.

In a commercial product, the development board would be replaced by a bare ESP32 module fed by a low-quiescent-current supply. With 3.3 V provided directly by the buck converter, the same light sleep operation would draw about 0.5 mA from the battery, eight times less than the 3.8 mA estimated here; the 0.2 mA of the voltage divider would then become significant and would have to be switched off between samples. The on-chip ADC would give way to an external converter or a digital temperature sensor, and the halogen lamps, which dominate the energy budget, to a more efficient heater.

5. CONCLUSIONS

This paper presented the firmware of an ESP32 microcontroller that implements closed-loop temperature control of a compact thermal chamber, using the on-chip ADC, the LEDC PWM peripheral and light sleep to combine control and low power consumption in a single device. Estimates based on the datasheet and a simulation of the control loop indicate that light sleep, with the LEDC clocked by `RTC8M_CLK` so that the PWM keeps running, reduces the battery current of the electronics from 62 to 3.8 mA and extends their runtime from 1.6 to 26 days, while the 60 s control period settles the temperature within ±0.5 °C of the setpoint after 14.7 min, with a steady-state standard deviation below 0.1 °C. At system level, however, the gain is limited to 13% while the lamps are on, and the main limits lie outside the microcontroller: the 6 mA overhead of the

development board, the systematic error of the ADC and the heater itself.

Future work includes validating these estimates on the prototype with a shunt resistor or power analyzer and a reference thermometer, calibrating the ADC, adopting a bare module with a low-quiescent-current supply and a switched voltage divider, using an adaptive control period (short during heating, long at steady state) and replacing the halogen lamps with a more efficient heater.

REFERENCES

[1] Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems, 2024. Available at: https://www.espressif.com.

[2] Espressif Systems. *ESP32 Technical Reference Manual*. Espressif Systems, 2024. Available at: https://www.espressif.com.

[3] Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, 5 edition, 2010.

[4] Vishay Siliconix. *IRLZ44N Power MOSFET*. Vishay Siliconix, 2020. Datasheet. Available at: https://www.vishay.com.

[5] Texas Instruments. *LM35 Precision Centigrade Temperature Sensors*. Texas Instruments, 2017. Datasheet. Available at: https://www.ti.com/product/LM35.

[6] Espressif Systems. *ESP-IDF Programming Guide: Analog to Digital Converter (ADC) and LED Control (LEDC)*. Espressif Systems, 2026. Available at: https://docs.espressif.com/projects/esp-idf.

[7] Karl J. Åström and Tore Hägglund. *PID Controllers: Theory, Design, and Tuning*. Instrument Society of America, 2 edition, 1995.

[8] Advanced Monolithic Systems. *AMS1117 1A Low Dropout Voltage Regulator*. Advanced Monolithic Systems. Datasheet.